Foveated Stereo Rendering in Blender: A Unified Sensor Framework

Luiz Velho
with Claude (Anthropic)
VISGRAF Lab, IMPA

Technical Report · course reading · draft 5 November 2026

Abstract

A camera samples a scene uniformly; an eye does not. This report describes `fov3d.rendering`, a framework that turns Blender's Cycles path tracer into a binocular, gaze-directed observer. Two eyes on a fixed head can each carry one of four sensors: a log-polar foveated camera, a perspective tangent-plane camera, an equirectangular panorama and a wide reference view. Every look, whatever the sensor, returns the same record: one ray per sample with its direction, radiance, solid-angle footprint and, as oracle, the surface it hit. The framework unifies two research code bases from the VISGRAF laboratory, fov-3d-vision [3] and f3d-vision [7].

The reader should come away with four things: why foveated, active sampling is worth simulating (Section 1); how the framework is layered (Section 2); the geometry and sampling mathematics behind each sensor and each measurement (Sections 3–7); and how to drive it from Python (Section 8). Implementation detail is kept at the level of modules and interfaces.

1 Motivation

Foveation is first a bandwidth decision and only then an attention mechanism: an observer that samples finely where it looks and coarsely elsewhere sees the same field with about fifty times fewer rays, and must then decide where to look.

1.1 The bandwidth argument

Take a field of 45° around the gaze and a foveal spacing of $s_0 = 0.05°$ per sample. A uniform sampler at that spacing needs about 2.5 million rays. The log-polar sensor of Section 4 covers the same field with 50,269 rays: **50.8× fewer**, as measured in fov-3d-vision's step A3 [5]. The saving comes from where the samples go: 16% of them land within 5° of the gaze, a cone that holds only 1.3% of the field's solid angle (0.19% of the full sphere).

The price is that most of the scene is seen coarsely at any moment. Fine detail must be collected by moving the gaze, so sensing becomes a sequence of decisions.

1.2 The active loop

Both code bases organise vision as a loop: **fixate** (render a binocular look), **reconstruct** (turn the fovea's matches into metric 3-D points), **update memory** (fuse them into a persistent map and mark the sphere of directions already seen) and **saccade** (choose the next gaze). Two results show what the loop buys:

- In fov-3d-vision, fifty foveated fixations of the Classroom scene measured 92% of a 60° field of regard at a median inverse-depth error of $0.04\,\mathrm{m}^{-1}$, with coverage-first, random and information-seeking policies within 12% of each other [6].
- In f3d-vision, a greedy explorer with perspective foveae saw 100.00% of the sphere around the head and reconstructed depth for 99.26% of it, in 356 fixations [7, 2].

1.3 Why a simulator with an oracle

A rendered scene knows, for every ray, the point it hit and the object that point belongs to. That oracle lets an experiment separate three sources of error that a real camera mixes together: the sensor, the stereo matcher and the gaze policy. A path tracer also produces physically plausible radiance, with noise whose level is set by the number of samples, so sensing cost can be controlled and measured (Section 7).

1.4 Why one framework

The two code bases grew apart. fov-3d-vision built the log-polar foveated camera and a spherical stereo field; f3d-vision built a 12° perspective pair with oracle correspondence, a persistent surfel map and a render server. Questions that cross the line, such as whether the log-polar sensor changes the explorer's coverage at equal rays, need both sensors behind one rig, one render configuration and one output record. That is what this framework provides, for any `.blend` scene.

Three working rules carry over from both projects and shape the design: every number is either measured or labelled an estimate; every component ships a check that can fail; and data the observer may see is kept apart from data only the evaluator may see.

2 Architecture

The framework is a stack of layers that every sensor shares, with only the camera swapped per look: a caller sends a request, one handler inside Blender renders it, and one record per eye comes back (Figure 1).

Each layer has one job:

- **Scene.** Opens any `.blend` file once per session and fixes the head at the scene's `EYE` object. It removes anything in the file that would override the framework's settings (an animated seed, extra render passes), numbers every renderable object so the Object Index pass can name it, and runs the sanity checks of Section 9.

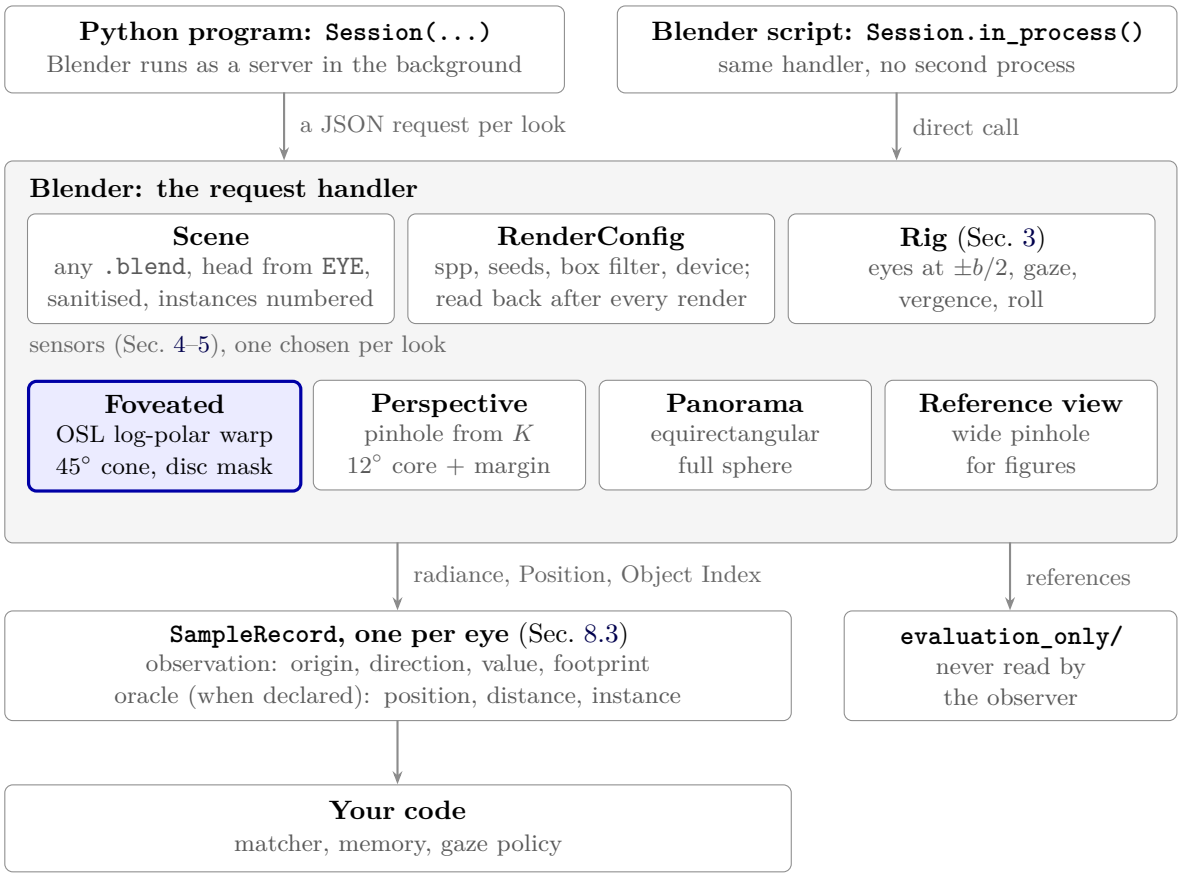


Figure 1: Architecture of `fov3d.rendering`: two entry points, one request handler, four sensors and one record per eye. The foveated sensor is highlighted because it is the one sensor no off-the-shelf camera provides.

- **RenderConfig.** One frozen set of Cycles settings for the session: samples per pixel, seeds, a box pixel filter, no adaptive sampling or denoising, the GPU back-end. After every render the settings Cycles actually used are read back and stored with the look (Section 7).
- **Rig.** Turns a gaze request into a world pose for each eye, with one formula for every sensor (Section 3).
- **Sensors.** Each sensor has two halves: a numpy *model* that maps pixels to rays and gives each pixel's footprint, and a Blender *builder* that creates the camera. Analysis code uses the model without Blender (Sections 4–5).
- **Record.** Converts the rendered passes into a `SampleRecord`, one entry per valid sample, with each field labelled as observation or oracle. References go to a separate `evaluation_only/` area (Section 8.3).
- **Execution.** A `Session` either runs Blender as a persistent background server fed one

JSON request per look, or, inside a Blender script, calls the same handler directly (Section 8.1).

Two properties follow from this layout. The record does not depend on the sensor, so downstream code (a matcher, a map, a gaze policy) can switch sensors without change. And the observer's view and the evaluator's truth are separated by construction, not by convention.

3 Coordinate frames and the binocular rig

All sensors share one rig: a fixed head with two eyes that rotate about their own centres, so every look is fully described by the head pose, which eye, a gaze direction and a roll convention.

3.1 Frames

- **World W.** Blender's frame, in metres, with $+Z$ up.
- **Head H.** The rigid frame of the scene's `EYE` object, fixed for a whole run: $+X$ to the right (along the baseline), $+Y$ up, $-Z$ forward. It is given by a rotation R_{WH} and an origin $\mathbf{o}$.
- **Eye camera E.** Blender's camera convention: the camera looks along its local $-Z$ with $+Y$ up and $+X$ right. Computer-vision code uses the OpenCV convention (x right, y down, z forward); the two differ by $C = \mathrm{diag}(1, -1, -1)$.

The eye centres lie on the head's X axis, a baseline b apart ($b = 63\,\mathrm{mm}$ by default):

$$\mathbf{c}_{L,R} = \mathbf{o} \mp \tfrac{b}{2}\, R_{WH}\, \hat{\mathbf{x}}. \tag{1}$$

3.2 Gaze

A gaze is a yaw ψ (positive to the right) and a pitch θ (positive up), measured in the head frame. The gaze direction is

$$\mathbf{d}(\psi, \theta) = (\sin\psi\cos\theta,\ \sin\theta,\ -\cos\psi\cos\theta), \tag{2}$$

and its inverse is $\psi = \mathrm{atan2}(d_x, -d_z)$, $\theta = \mathrm{atan2}\big(d_y, \sqrt{d_x^2 + d_z^2}\big)$. The same formula, read with ψ as longitude and θ as latitude, is the panorama's pixel-to-direction map (Section 5.2), which is why every sensor agrees on what "straight ahead" means.

The world pose of an eye camera is one composition, used by every sensor:

$$M_{\mathrm{eye}} = T(\mathbf{o})\; R_{WH}\; T\big(\pm\tfrac{b}{2}\hat{\mathbf{x}}\big)\; R_y(-\psi)\; R_x(\theta)\; R_{\mathrm{roll}}. \tag{3}$$

Applying $R_y(-\psi)\, R_x(\theta)$ to the camera's forward axis $(0, 0, -1)$ gives exactly $\mathbf{d}(\psi, \theta)$: yaw is applied after pitch, so the elevation of the gaze equals θ. The head never translates; only the eyes rotate.

3.3 Vergence

A binocular look fixates one point $\mathbf{P}$. Each eye's gaze is the unit vector from its centre to $\mathbf{P}$, expressed in the head frame and converted to (ψ_i, θ_i) by the inverse above:

$$\mathbf{d}_i = R_{WH}^{\top} \frac{\mathbf{P} - \mathbf{c}_i}{\|\mathbf{P} - \mathbf{c}_i\|}, \qquad i \in \{L, R\}. \tag{4}$$

The vergence angle is $\gamma = \arccos(\mathbf{d}_L \cdot \mathbf{d}_R)$. For a point straight ahead at distance D it is $\gamma = 2\arctan(b/2D)$; at the 2.10 m used by the perspective experiments that is 1.72°. A look can be requested three ways: by a world point $\mathbf{P}$, by a cyclopean gaze plus a distance D (then $\mathbf{P} = \mathbf{o} + D\,\mathbf{d}$), or by giving each eye its own (ψ, θ).

3.4 Roll

A gaze direction fixes the camera's forward axis but not its rotation about that axis. The framework offers two conventions, and each fails in a different place:

- **Upright** (zero torsion): the image x axis is kept in the head's horizontal plane, $\hat{\mathbf{x}} = \text{normalize}(\hat{\mathbf{z}} \times \hat{\mathbf{y}}_H)$, with $\hat{\mathbf{z}}$ the forward axis in OpenCV convention. It is undefined when looking straight up or down.
- **Baseline-projected**: the head's baseline direction $\mathbf{h} = \hat{\mathbf{x}}_H$ is projected into the image plane, $\hat{\mathbf{x}} = \text{normalize}\big(\mathbf{h} - (\mathbf{h} \cdot \hat{\mathbf{z}})\,\hat{\mathbf{z}}\big)$. It is undefined only when looking along the baseline, where binocular depth is undefined anyway, which makes it the right choice for stereo all around the sphere.

For a verged pair the fixation's epipolar plane contains the baseline and both gaze axes, so baseline-projected roll keeps that plane on the central image row of both eyes. The foveated sensor's warp is circularly symmetric, so roll only rotates its disc and does not change what it measures; the perspective pair's rectification depends on it. Roll is therefore a property of the rig, recorded with every look, not of the sensor.

4 The foveated log-polar warp

The foveated sensor is a Cycles camera, written in OSL, that maps a square raster onto a cone of directions so that the spacing between samples grows linearly with eccentricity. One parameter, E_2, sets the budget; the field's half-angle $e_{\max}$ costs only logarithmically.

4.1 The spacing law

Let e be the angle between a ray and the gaze. The sensor places samples with spacing

$$s(e) = s_0 \left(1 + \frac{e}{E_2}\right), \tag{5}$$

where s_0 is the foveal spacing and E_2 is the eccentricity at which the spacing has doubled. This is the linear form used to describe cortical magnification in primate vision. The framework uses $E_2 = 2°$ and $e_{\max} = 45°$; fov-3d-vision found its disparity objective flat in E_2 (decision D16 [4]), so E_2 is chosen for cost, not accuracy.

4.2 From the spacing law to the raster

Walking outward from the centre, each pixel advances the eccentricity by one spacing, $de/dw = s(e)$. Integrating gives the raster radius, in pixels, at which eccentricity e is reached:

$$w(e) = \int_0^e \frac{de'}{s(e')} = \frac{E_2}{s_0}\,\ln\!\left(1 + \frac{e}{E_2}\right). \tag{6}$$

The raster must reach $e_{\max}$ at its rim, so it is $N = 2\,w(e_{\max})$ pixels across:

$$N = 2\,\frac{E_2}{s_0}\,\ln\!\left(1 + \frac{e_{\max}}{E_2}\right). \tag{7}$$

Writing $r = w(e)/w(e_{\max}) \in [0,1]$ for the normalised radius and inverting gives the mapping the shader evaluates:

$$e(r) = E_2\left[\left(1 + \frac{e_{\max}}{E_2}\right)^{r} - 1\right]. \tag{8}$$

s_0 has dropped out: the shader needs only E_2 and $e_{\max}$, and the foveal spacing is set by the resolution chosen, $s_0 = 2E_2\ln(1 + e_{\max}/E_2)/N$. The two profiles are `small` ($s_0 = 0.1°$, $N = 126$) and `full` ($s_0 = 0.05°$, $N = 253$).

Pixel to ray. For a raster position $(u, v) \in [0,1]^2$, let $\mathbf{p} = (u - \frac{1}{2}, v - \frac{1}{2})$, $r = 2\|\mathbf{p}\|$ and $\varphi = \operatorname{atan2}(p_y, p_x)$. In the shader's frame ($+Z$ forward) the ray is

$$\mathbf{d} = (\sin e\cos\varphi,\ \sin e\sin\varphi,\ \cos e), \qquad e = e(r). \tag{9}$$

Pixels with $r > 1$ receive zero throughput. They are masked by the analytic test $r \le 1$, never by alpha, which Cycles leaves at 1 even where no radiance arrives.

4.3 Footprint and anisotropy

Let $\rho = rN/2$ be the radius in pixels. Along the radius, consecutive pixels are one spacing $s(e)$ apart by construction. Around the circle they are $\sin(e)/\rho$ apart, which is smaller, so peripheral pixels are elongated radially. The solid angle one pixel covers is the product:

$$\omega(\rho) = \frac{\sin e}{\rho}\,\frac{de}{d\rho}, \qquad \frac{de}{d\rho} = \frac{2}{N}\,\ln\!\left(1 + \frac{e_{\max}}{E_2}\right)(e + E_2). \tag{10}$$

At the centre both factors tend to s_0, so $\omega \to s_0^2$. At the rim of the `full` profile the radial spacing is 1.17° and the tangential spacing 0.32°, an elongation of 3.7, and one pixel covers about $151\,s_0^2$. Because spacing alone does not describe a non-square pixel, the record carries each sample's footprint ω (Section 7.2).

4.4 What the warp saves

The disc holds about $\pi N^2/4$ samples: 50,269 at the `full` profile. A uniform sampler needs about Ω/s_0^2 samples for the cone of solid angle $\Omega = 2\pi(1 - \cos e_{\max}) = 1.84\,\mathrm{sr}$, which estimates 2.42 million; counting the pixels of a uniform raster at s_0, fov-3d-vision measured

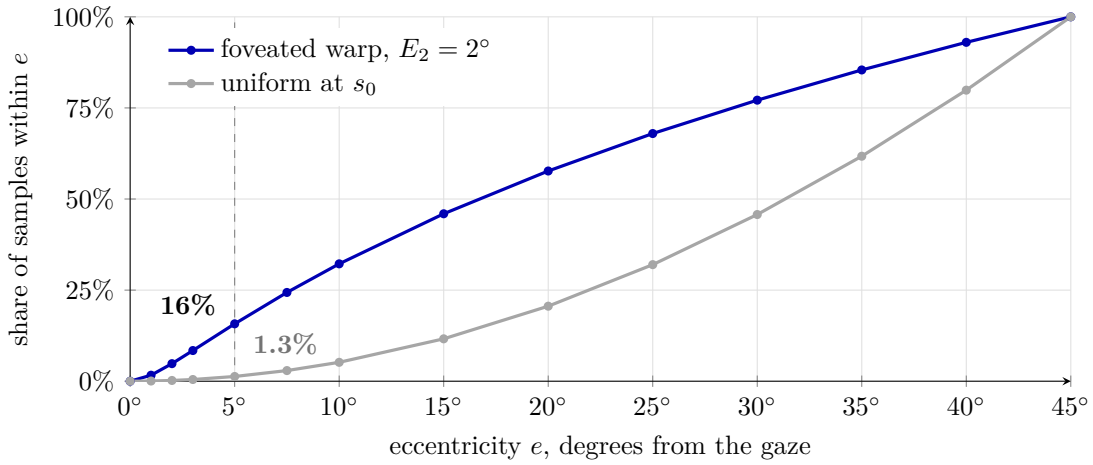


Figure 2: Within 5° of the gaze the foveated warp places 16% of its samples, uniform sampling 1.3%. Cumulative share of samples inside eccentricity e over a 45° field, computed from Eqs. (6)–(8).

2,553,563, hence the 50.8× of Section 1. The ratio does not depend on s_0, since both counts scale as $1/s_0^2$.

The share of samples inside eccentricity e is $r(e)^2$ for the warp and $(1-\cos e)/(1-\cos e_{\max})$ for uniform sampling (Figure 2). The warp puts 16% of its samples within 5° of the gaze, twelve times the uniform share. Because the raster size grows with $\ln(1+e_{\max}/E_2)$, widening the field is cheap: from 20° to 45° costs 1.7× in raster area, and from 45° to 80° a further 1.4×.

5 Perspective and panoramic sensors

The other three sensors are standard Blender cameras described by the same three quantities as the warp: a pixel-to-ray map, a footprint per pixel and a valid region. That shared description is what lets one record hold any of them.

5.1 The perspective tangent pair

Each eye is a pinhole camera on the plane tangent to the sphere of directions at the gaze. Its square raster has a **core** that spans 12° and a **margin** of $\frac{3}{4}$ of the core on every side:

$$n = c + 2 \cdot \tfrac{3}{4}c, \qquad f = \frac{c}{2\tan 6°}, \qquad K = \begin{pmatrix} f & 0 & \frac{n-1}{2} \\ 0 & f & \frac{n-1}{2} \\ 0 & 0 & 1 \end{pmatrix}, \tag{11}$$

with core $c = 128$ pixels (`small`) or 256 (`full`). At `full` that gives $n = 640$, $f = 1217.8$ pixels, a raster about 29.4° across and a spacing of $1/f = 0.047°$ at the centre, close to the warp's $s_0 = 0.05°$. Blender is given the same camera through its lens length, lens $= f \cdot 36\,\text{mm}/n$ on a 36 mm sensor.

In the OpenCV frame the ray through pixel (u, v), counted from the top-left pixel centre, is

$$\mathbf{d} \propto K^{-1}(u, v, 1)^{\top} = \left(\frac{u - c_x}{f},\ \frac{v - c_y}{f},\ 1 \right), \tag{12}$$

and the matrix C of Section 3.1 converts it to Blender's camera frame. A pixel at offset $(x, y) = (u - c_x, v - c_y)$ from the principal point covers the solid angle

$$\omega(x, y) = \frac{f}{(f^2 + x^2 + y^2)^{3/2}}, \tag{13}$$

which is $1/f^2$ at the centre and only 18% smaller at the corner: unlike the warp, this sensor is nearly uniform over its small field. The margin exists for the stereo matcher, which rectifies the pair and searches for correspondences there; only measurements in the central core of the rectified raster are accepted.

5.2 The equirectangular panorama

The panorama sees the whole sphere from an eye centre. For a $W \times H$ raster with $H = W/2$ and normalised pixel centre (u, v), $v = 0$ at the top row,

$$\lambda = (u - \tfrac{1}{2})\, 2\pi, \quad \beta = (\tfrac{1}{2} - v)\, \pi, \quad \mathbf{d} = (\sin\lambda \cos\beta,\ \sin\beta,\ -\cos\lambda \cos\beta), \tag{14}$$

which is the gaze formula (2) with longitude λ as yaw and latitude β as pitch: the panorama's centre pixel is the head's primary gaze. Each pixel covers

$$\omega(\beta) = \frac{2\pi}{W} \frac{\pi}{H} \cos\beta, \tag{15}$$

and the footprints sum to 4π sr, a check every panorama passes (Section 9). The panorama has four uses: the uniform reference at foveal spacing ($W = 7200$ gives 0.05° at the equator), the noise-floor measurement, the post-hoc evaluation sphere (720×360, 0.5° cells) and, rendered from each eye centre, the per-eye truth for stereo. An optional second render overrides every material with its back-face flag, so a scene whose set dressing is seen from behind can be rejected.

5.3 The sensors side by side

The reference view is the perspective model of Section 5.1 with a wide field (half-angle 27° horizontally) and no margin; it renders what a reader of a figure expects to see.

Sensor	Rays per eye at `full`	Spacing at the gaze	Field	Used for
foveated	50,269	0.05°	45° half-angle cone	fixations in the active loop
perspective	409,600 (65,536 in the core)	0.047°	12° core, 29.4° raster	local stereo with oracle correspondence
panorama	25.9 million (7200×3600)	0.05° at the equator	full sphere	references, noise floor, evaluation
reference view	4.9 million (2400×2048)	0.024°	54° wide	figures

Table 1: The four sensors at the `full` profile.

6 From pixels to geometry

A binocular look becomes 3-D geometry through two angles per matched ray pair: the epipolar plane they share and the parallax between them. Inverse depth is then, to first order, proportional to the parallax, and that fact sets both the measurement and its uncertainty.

6.1 What a render returns

Each pixel carries three passes: **Combined**, the radiance L the eye sees; **Position**, the world point $\mathbf{X}$ where the pixel's ray first hit a surface; and **Object Index**, the instance k of that surface (0 for a miss). From them the record derives the range and checks the sensor model:

$$r = \|\mathbf{X} - \mathbf{c}\|, \qquad \mathbf{X} \approx \mathbf{c} + r\,\mathbf{d}, \tag{16}$$

where $\mathbf{c}$ is the eye centre and $\mathbf{d}$ the ray the sensor model predicts for that pixel. Range is computed from Position rather than read from Cycles' Depth pass, because Depth is the ray length for panoramic and custom cameras but need not be for a perspective one; Position means the same thing for every sensor.

Position and Object Index are **oracle** data: they describe the first surface hit, which a real sensor cannot observe. An observer that is meant to learn from images reads only radiance and the sensor model; an experiment that declares oracle correspondence may also read the oracle fields (Section 8.3).

6.2 Oracle correspondence

The oracle answers the hardest question in stereo, which pixels match. For a left pixel with first hit $\mathbf{X}$, the right eye sees $\mathbf{X}$ along $\mathbf{d}_R = (\mathbf{X} - \mathbf{c}_R)/\|\mathbf{X} - \mathbf{c}_R\|$; the sensor model inverts that direction into a right pixel. The match is accepted only if the right eye's own Position at that pixel equals $\mathbf{X}$ within a tolerance. If it does not, $\mathbf{X}$ is hidden from the right eye by something nearer, and the left pixel is half-occluded. This is how f3d-vision's explorer gets "perfect" stereo, and how fov-3d-vision measures the error of a real matcher against truth.

6.3 Epipolar coordinates on the sphere

With a fixed head, every epipolar plane contains the baseline, the head's X axis. A head-frame direction $\mathbf{d}$ is therefore best described by two angles:

$$\vartheta = \arccos(d_x), \qquad \varphi = \operatorname{atan2}(d_y,\, -d_z). \tag{17}$$

ϑ is the angle from the baseline and φ picks the epipolar plane, a rotation about the baseline. Two rays that see the same point share φ, and their parallax is $\delta = \vartheta_R - \vartheta_L > 0$. Unlike image disparity, these coordinates are defined over the whole sphere, so they work for every sensor and every gaze.

6.4 Triangulation and inverse depth

In the triangle formed by the two eye centres and the point, the law of sines gives the distance from the left eye exactly:

$$r_L = b\,\frac{\sin\vartheta_R}{\sin(\vartheta_R - \vartheta_L)}, \qquad \mathbf{X} = \mathbf{c}_L + r_L\,\mathbf{d}_L. \tag{18}$$

For small parallax, $\sin\delta \approx \delta$ and $\vartheta_R \approx \vartheta_L = \vartheta$, so inverse depth is linear in parallax:

$$\rho = \frac{1}{r} \approx \frac{\delta}{b\,\sin\vartheta}, \qquad \sigma_\rho \approx \frac{\sigma_\delta}{b\,\sin\vartheta}, \qquad \sigma_r \approx r^2\,\sigma_\rho. \tag{19}$$

This is why the stereo field measures **inverse depth** with a variance: its error is the parallax error scaled by a constant. A worked example: with $b = 63\,\mathrm{mm}$, a point straight ahead ($\vartheta = 90°$) and a parallax error of one foveal sample, $\sigma_\delta = 0.05°$, the inverse-depth error is $0.014\,\mathrm{m}^{-1}$, which at $2.7\,\mathrm{m}$ is $\pm 10\,\mathrm{cm}$. That is the scale fov-3d-vision measured where the fovea had looked (0.015–$0.019\,\mathrm{m}^{-1}$, 10–14 cm at 2.7 m [6]). Depth precision falls with the square of distance, and nothing can be measured along the baseline, where $\sin\vartheta = 0$.

When the two rays come from a real matcher they rarely meet exactly. The framework then takes the midpoint of their closest approach: with $\mathbf{w} = \mathbf{c}_L - \mathbf{c}_R$ and $\kappa = \mathbf{d}_L \cdot \mathbf{d}_R$,

$$s = \frac{\kappa\,(\mathbf{d}_R\cdot\mathbf{w}) - \mathbf{d}_L\cdot\mathbf{w}}{1-\kappa^2}, \quad t = \frac{\mathbf{d}_R\cdot\mathbf{w} - \kappa\,(\mathbf{d}_L\cdot\mathbf{w})}{1-\kappa^2}, \quad \mathbf{X} = \tfrac{1}{2}\big[(\mathbf{c}_L + s\,\mathbf{d}_L) + (\mathbf{c}_R + t\,\mathbf{d}_R)\big], \tag{20}$$

and the gap between the two closest points is a useful check on the match. What happens to the points afterwards (fusion into a surfel map at 12 mm, the coverage sphere, the gaze policy) is the business of the code that consumes the record, not of the rendering framework.

7 Sampling and noise

Every pixel the framework returns is a Monte Carlo estimate of the average radiance over that pixel's footprint. Its noise falls as one over the square root of the samples per pixel, and the settings are chosen so that this statement stays exactly true.

7.1 Monte Carlo error

Cycles estimates a pixel by averaging n independent light paths, $v = \frac{1}{n}\sum_j L_j$. The estimate is unbiased and its standard deviation falls as

$$\sigma_v(n) = \frac{\sigma_1}{\sqrt{n}}, \tag{21}$$

where σ_1 is the spread of a single path, which depends on the scene's lighting and materials. Sensing cost is therefore controlled by one number, the samples per pixel (spp): 64 for a fixation at `small`, 256 at `full`. Two Cycles features would break the $1/\sqrt{n}$ law and are always off for measurement: adaptive sampling, which stops some pixels early, and denoising, which mixes neighbouring pixels.

7.2 The box filter makes a pixel a footprint average

With a box pixel filter of width 1, every path for a pixel starts at a uniformly random point inside that pixel's square on the raster, and contributes only to that pixel. Through the sensor's pixel-to-ray map the square becomes the pixel's footprint Ω_i on the sphere, so the pixel estimates

$$v_i \approx \frac{1}{\omega_i}\int_{\Omega_i} L(\mathbf{d})\, d\Omega. \tag{22}$$

The approximation is only that sampling is uniform over the raster square rather than over solid angle, which differs little within one pixel. This is why each record entry carries its footprint ω_i: a foveated sample covering $151\, s_0^2$ in the periphery is compared with a reference by averaging the reference over the same patch of sphere. Wider filters, such as Blackman–Harris, spread a path over neighbouring pixels and break this identity; they are used only for previews.

7.3 References must be quieter than what they judge

A reference that is as noisy as the fixation it judges would hide the fixation's error in its own. fov-3d-vision's decision D7 [4] requires the reference's relative noise to be at most a third of the fixation's. With equal per-path spread that means

$$\sqrt{\frac{n_{\text{fix}}}{n_{\text{ref}}}} \le \frac{1}{3} \quad\Longleftrightarrow\quad n_{\text{ref}} \ge 9\, n_{\text{fix}}. \tag{23}$$

The profiles exceed it: 1024 against 64 spp at `small` (16×), 8192 against 256 at `full` (32×). The price is time. A `full` reference panorama took 32–36 minutes on an RTX 4090, against about 270 ms for a foveated pair, so references are rendered once per scene, pinned by checksum and kept.

7.4 Seeds

The random sequence of a render is set by its seed. The two eyes always use different seeds (2111 and 2112 by default) so their noise is independent; identical noise in both eyes

would look like a perfect match and bias the stereo. Blender files can animate the seed (the Classroom does), so the framework removes such animation before rendering and checks after each render that the seed it set is the one used.

Two renders of the same look with different seeds measure the noise directly. Their difference has variance $2\sigma^2$, so

$$\hat{\sigma}^2 = \tfrac{1}{2}\,\mathrm{Var}\Big(v^{(s)} - v^{(s')}\Big)\,. \tag{24}$$

This noise floor sets the thresholds of the framework's checks, rather than a fixed constant (decision D10 [4]). One limit applies: on OptiX the same seed does not give bit-identical images, so two conditions that must see identical noise are run on one saved observation rather than two renders.

8 Using the framework

A program opens a `Session` on a scene, chooses a sensor and a gaze, and asks for a `look`; it gets back one `SampleRecord` per eye. Everything else in this section is a variation on those three calls.

8.1 A first look

```
from fov3d.rendering import Session, RenderConfig, Rig, Gaze, sensors

cfg = RenderConfig(profile="small", device="OPTIX")      # build on small, report on full
rig = Rig(ipd_m=0.063, roll="upright")

with Session("scenes/classroom/classroom_eye.blend", cfg, rig=rig) as s:
    eye = sensors.Foveated(E2_deg=2.0, e_max_deg=45.0)      # s0 comes from the profile
    look = s.look(eye, eyes=("L", "R"), gaze=Gaze.verged_point([0.4, 2.1, 1.1]))
    L, R = look.records
    print(L.n_samples, L.direction.shape, L.footprint_sr.sum())
```

`Session` starts Blender in the background, opens the scene once and keeps it open; each `look` is one request to it, and the context manager stops it. Inside a script that already runs in Blender, `Session.in_process(cfg, rig=rig)` gives the same object without a second process.

8.2 The main objects

Each sensor's `.model` answers the geometric questions of Sections 4–5 in plain numpy, so analysis code can use it on any machine:

```
m = sensors.Foveated(E2_deg=2.0, e_max_deg=45.0).model(profile="full")
m.raster_size                  # (253, 253)
dirs, omega, valid = m.rays()  # camera-frame directions, footprints (sr), disc mask
uv = m.pixel_of(dirs[valid])   # inverse map, used by oracle correspondence
```

Object	Made by	What it holds or does
`RenderConfig`	`RenderConfig(profile, device, device_policy, seed_policy, color, exr_codec)`	the frozen Cycles settings of Section 7; `small` or `full` fixes spp and spacing
`Rig`	`Rig(ipd_m, roll)`	baseline and roll convention (Section 3); `roll` is `"upright"` or `"baseline_projected"`
`Gaze`	`Gaze.yaw_pitch(psi, theta, vergence_m=2.1)`, `Gaze.verged_point(P)`, `Gaze.per_eye(L=(psi, theta), R=(psi, theta))`	where the eyes point, in degrees and metres (Sections 3.2–3.3)
sensors	`sensors.Foveated(E2_deg, e_max_deg, s0_deg=None)`, `sensors.Perspective(core_fov_deg=12)`, `sensors.Panorama(preset="preview360")`, `sensors.ReferenceView(width, height, half_fov_x_deg)`	the camera model; `.model` works without Blender
`Session`	`Session(blend, cfg, rig=rig, access="observation")`	the scene, the render server and the truth boundary
`Look`	`session.look(sensor, eyes, gaze, spp=None, seeds=None)`	the request, `meta`, and `.records`
`SampleRecord`	inside a `Look`, or `SampleRecord.load(path)`	one eye's samples (Section 8.3)

Table 2: The objects of the Python API.

8.3 The record and the truth boundary

A `SampleRecord` holds one entry per valid sample. Its observation fields are `origin` (the eye centre), `direction`, `value` (linear RGB radiance), `footprint_sr`, `raster_index`, and the labels `eye`, `look_id` and `sensor`. Its oracle fields are `position_w`, `distance_m` (infinite on a miss) and `instance_id` (0 on a miss). Helpers convert between views: `rec.as_raster("value")` rebuilds the image, and `rec.direction_head` and `rec.epipolar()` give head-frame directions and the angles (ϑ, φ) of Section 6.3.

The session's `access` decides who may see what. With `access="observation"`, the default, oracle fields are not loaded and reading one raises an error, so a learning observer cannot cheat by accident. An experiment that declares oracle correspondence or segmentation opens the session with `access="oracle"`, and that choice is written into every look's `meta.json`. References are a third tier: `session.reference(eye="L")` renders the panorama of Section 5.2 into an `evaluation_only/` folder that an observer session refuses to open.

8.4 Worked example: inverse depth from oracle stereo

This example implements Sections 6.2–6.4 on one perspective pair.

```
import numpy as np

with Session(blend, RenderConfig(profile="full"), rig=Rig(roll="baseline_projected"),
             access="oracle") as s:
    look = s.look(sensors.Perspective(), eyes=("L", "R"),
                  gaze=Gaze.yaw_pitch(10, -5, vergence_m=2.1))
    L, R = look.records
    iL, iR = look.oracle_matches(core_only=True)   # visible in both eyes (Sec. 6.2)
    thL, phL = L.epipolar(iL)
    thR, phR = R.epipolar(iR)
    assert np.allclose(phL, phR, atol=1e-6)        # same epipolar plane
    rL = look.rig.baseline_m * np.sin(thR) / np.sin(thR - thL)   # law of sines (Sec. 6.4)
    err = rL - L.distance_m[iL]    # zero up to round-off: a check, not a result
```

Replacing `look.oracle_matches` with a real matcher's output, and keeping `L.distance_m` only for scoring, turns the same lines into a stereo evaluation.

8.5 Worked example: an active loop

The framework renders; the loop's memory and policy are the student's. A minimal loop:

```
with Session(blend, RenderConfig(profile="small"), access="oracle") as s:
    eye = sensors.Foveated(E2_deg=2.0, e_max_deg=45.0)
    gaze, memory = Gaze.yaw_pitch(0, 0, vergence_m=2.0), MyMemory()
    for k in range(50):
        look = s.look(eye, eyes=("L", "R"), gaze=gaze)
        memory.update(measure(look))     # fixate -> reconstruct -> update memory
        gaze = my_policy(memory)         # saccade
        if gaze is None:                 # nothing left worth looking at
            break
```

Because every sensor returns the same record, changing `eye` to `sensors.Perspective()` is the only edit needed to compare the two sensors in the same loop. Compare them at equal cost: `eye.model(profile).rays_per_eye` gives 50,269 rays for the foveated sensor and 409,600 for the perspective one at `full`, so equal rays means about eight foveated looks per perspective look.

8.6 Requests and the command line

Every `look` is serialised as a JSON request (schema `fov3d-render-request-v1`) holding the sensor and its parameters, the eyes, the gaze, the roll, spp, seeds, passes and the output folder. Each look's folder holds the request, a `meta.json` with the poses, the settings Cycles actually used and the timings, the raw EXR per eye and the records. The same requests can be issued from a shell:

```
fov3d-render look --blend scene.blend --sensor foveated --eyes L R \
    --point 0.4 2.1 1.1 --profile full --out runs/demo/f0000
fov3d-render panorama --blend scene.blend --preset preview360 --eye L \
    --profile small --out previews/ref_L
fov3d-render check --blend scene.blend      # the checks of Sec. 9 on a new scene
```

9 Validation

Every layer ships a check with a known answer and a control: a deliberately broken input that the check must reject, because a check that cannot fail tests nothing. A new scene is accepted only when all checks pass on it (`fov3d-render check`). Table 3 lists them.

Check	Layer	Known answer	Control that must fail
Settings readback	configuration	samples, seed, adaptive sampling and time limit equal the request after every render	animate `cycles.samples` in the file
Scene sanity	scene	no holes or back-faces seen from the eye; floor and ceiling at the expected depth below and above the eye	a scene with a wall removed
Pose agreement	rig	the three historical pose formulas agree to 10^{-9} on a grid of gazes	flip the yaw sign in one
Roll frame	rig	baseline-projected roll keeps image $+X$ on the baseline at extreme gazes	upright roll at pitch $-89.9°$
Warp	foveated	rendered ray directions match the numpy model of Section 4.2	perturb E_2 by 1%
Footprint sum	every sensor	foveated: the cone's solid angle $2\pi(1-\cos e_{\max})$; panorama: 4π	drop the $\cos\beta$ factor
Projection	perspective	world points project to their pixels within 0.002 pixel	lens off by 1%
Ray-cast truth	perspective, panorama	Position within 0.02 mm of an independent ray cast	move the eye 1 mm
Equirect convention	panorama	Position directions match (λ, β) of Section 5.2	swap u and v
Range source	record	range from Position equals the Depth pass where Depth is a ray length	compare with a perspective Depth pass
Instance ids	record	Object Index equals the ray-cast object at sampled pixels	renumber an object after rendering
Noise and seeds	engine	the two eyes use different seeds; noise falls as $1/\sqrt{n}$	render both eyes with one seed
Loud failure	server	a failing request writes `fail-NNNN.json` and the process exits 1	catch the error without exiting

Table 3: Known-answer checks and their controls.

The check thresholds come from the measured noise floor of Section 7.4 and from the scene's contrast, not from fixed constants. All checks except the noise floor are designed to run in under ten seconds on the `small` profile.

10 Limitations and further reading

The framework simulates a controlled observer, not a real one; its limits are the price of an exact oracle.

- **Fixed head.** Eyes rotate, the head does not translate, so surfaces hidden from the head origin are never targets. Self-occluded surfaces need head motion, which fov-3d-vision tested only as a prescribed lateral move (FSG7a).
- **First-hit truth.** Position and Object Index describe the first surface a ray meets. For glass, cut-out alpha, volumes or emitters this differs from what the pixel shows; such objects are flagged when a scene is opened, and each experiment decides whether to exclude them.
- **Static, synthetic scenes.** No moving objects, no sensor model beyond path-traced radiance at a chosen spp, no real camera.
- **No physiological torsion.** Roll follows one of two engineering conventions (Section 3.4), not the torsion of a real eye.
- **Back-ends and repeatability.** The OSL camera is verified on OptiX and on CPU; other GPU back-ends are untested. OptiX renders are not bit-reproducible, so exact pairing of conditions reuses saved observations.
- **Reference cost.** Uniform references at foveal spacing take about half an hour per scene at the `full` profile, so they are rendered once and kept.

Further reading. The decisions behind the numbers in this report, each with what would overturn it, are in fov-3d-vision's `DECISIONS.md` [4]. The warp's derivation and its first measurement are in step A3 [5], and the active loop's results in the Phase C summary [6]. The perspective explorer's architecture is documented in f3d-vision [8]. The sequence of reports from the laboratory is summarised in TR-09-2026 [1] and TR-11-2026 [2].

References

[1] Luiz Velho and Claude. "Experiments on Active 3D Vision". Technical Report TR-09-2026, VISGRAF Lab – IMPA, 2026.

[2] Luiz Velho and Claude. "Active Foveal 3-D Vision Playground". Technical Report TR-11-2026, VISGRAF Lab – IMPA, 2026.

[3] VISGRAF Lab. *fov-3d-vision*: foveated omnidirectional rendering and 3D reconstruction. https://github.com/visgraf/fov-3d-vision

[4] VISGRAF Lab. fov-3d-vision, `DECISIONS.md`. https://github.com/visgraf/fov-3d-vision/blob/main/DECISIONS.md

[5] VISGRAF Lab. fov-3d-vision, step A3: the foveated camera. https://github.com/visgraf/fov-3d-vision/blob/main/docs/a3-foveated-camera.md

[6] VISGRAF Lab. fov-3d-vision, Phase C summary. https://github.com/visgraf/fov-3d-vision/blob/main/docs/phase-c-summary.md

[7] VISGRAF Lab. *f3d-vision*: the Active Foveal Playground. https://github.com/visgraf/f3d-vision

[8] VISGRAF Lab. f3d-vision, Greedy Foveal Playground architecture. https://github.com/visgraf/f3d-vision/blob/main/docs/architecture/greedy-foveal-playground.md